رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات



CIRP - System Analysis & Design

Customer Intelligence & Retention Platform

Project	Customer Churn Prediction & Segmentation Platform (CIRP)
Track	DEPI - AI & Machine Learning
Project Leader	Mohamed Atef Mohamed Abdo
Version / Date	v1.0 - November 2026
Repository	https://github.com/nhahub/NHA-5-011

1. Problem and Objectives

1.1 The problem

The shop in this project sells online and has three years of orders. Today it finds out that a customer has left only after a bad quarter is already over. Marketing also treats every customer the same, so money is spent on people who would have stayed anyway.

The data has no names, no complaints and no marketing history. Everything must be learned from buying behaviour: how recent the last order was, how often the customer buys, how much they spend and how that is changing.

1.2 Objectives

Table 1. Objectives and checks

ID	Objective	How we check it
O1	Predict which customers will stop buying in the next 90 days.	PR-AUC 0.80 or more and recall 0.90 or more on the test split.
O2	Explain each score in plain words.	Three reasons per customer on the simulator page.
O3	Group customers into six useful segments.	Each segment has a name and one suggested action; Silhouette 0.30 or more.
O4	Put both models in one simple dashboard.	One score in 2 seconds; 5,253 rows in 30 seconds; app start in 10 seconds.
O5	Make the results reproducible.	Same numbers, within 0.01, on a clean clone with seed 42.

1.3 The main numbers

- 22,372 customer records from 5,253 customers, with five snapshots from Sep 2010 to Sep 2011.
- 61.0% of the records are churn, so the classes are not balanced.
- Training: 12,178 rows (three snapshots). Validation: 4,941 rows (June 2011). Test: 5,253 rows (Sep 2011).
- Churn means no valid purchase in the 90 days after the snapshot date.

2. Use Case Diagram

The five use cases at the top of the diagram happen inside the dashboard. The three at the bottom happen offline, when the team prepares the data or the models. A normal user cannot start training from the app.

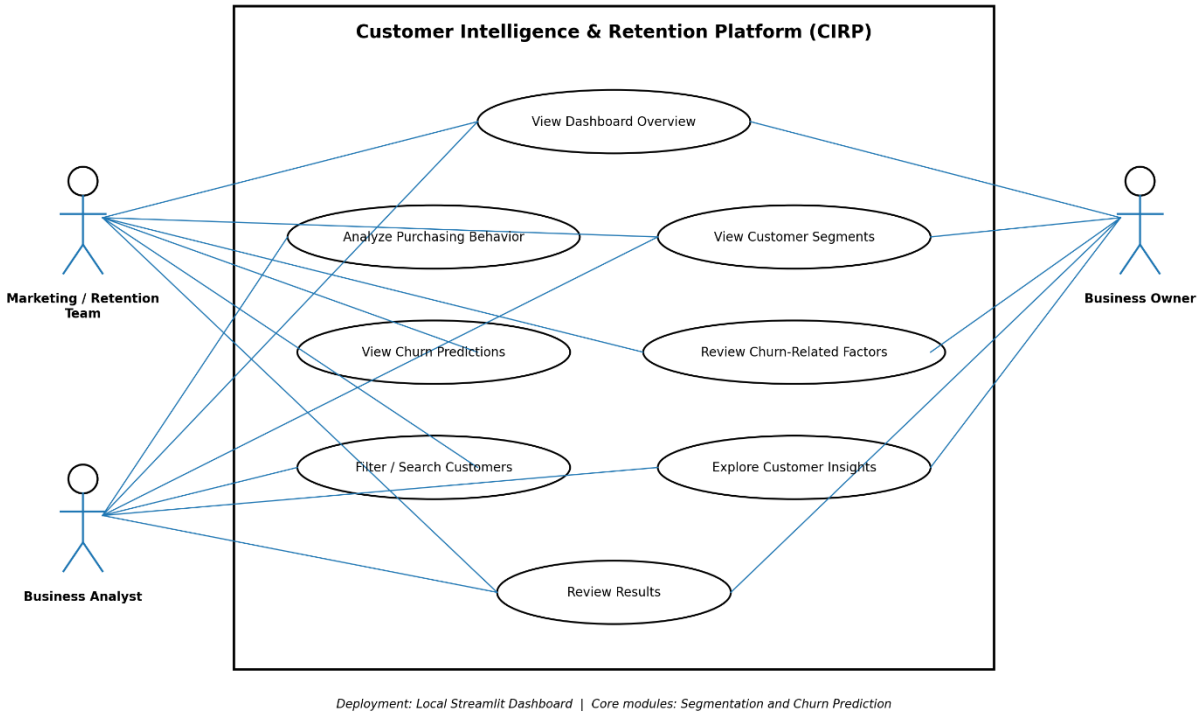


Figure 1. Use case diagram

Table 2. Use cases

ID	Use case	Actor	Output
UC-01	Score one customer	Marketing specialist	Risk, level, top three reasons, segment
UC-02	View segments	Marketing specialist	Six cards with size, churn rate and action
UC-03	Batch scoring	Retention manager	Scored table, KPIs and CSV download
UC-04	View overview	Business management	Four KPIs and charts
UC-05	View model performance	ML engineer	Metrics, curves and data dictionary
UC-06	Prepare the data	Data engineer	22 features, labels and splits
UC-07	Train the model	ML engineer	Model file, threshold and metrics
UC-08	Compare deep learning	ML engineer	Report and a keep-or-replace decision

3. Requirements in Short

3.1 Functional requirements

The requirements document lists all 36 requirements with their priority. This table shows what each module must do.

Table 3. Functional requirements by module

Module	IDs	What it does
Data	FR-D1 to D7	Reads both sheets, removes duplicates and cancellations, builds five snapshots, 22 features, the churn label and the time split.
Prediction	FR-P1 to P7	Compares three models, tunes the Random Forest, chooses the threshold on validation, saves one model file and scores test once.
Segmentation	FR-S1 to S5	Compares K-Means and DBSCAN, keeps six segments, transforms and scales the features, writes names and actions.
Application	FR-A1 to A8	Five pages: overview, segments, simulator, batch and performance, with input checks and CSV export.

Module	IDs	What it does
Integration	FR-I1 to I4	One shared feature list, read-only model loading, README steps and a fixed seed.
Deep learning	FR-ML1 to ML5	Isolated comparison of 30 configurations, selection on validation, one test run, written decision.

3.2 Non-functional requirements

Table 4. Non-functional requirements

ID	Requirement
NFR-1	One score in 2 seconds or less; 5,253 rows in 30 seconds or less; app start in 10 seconds.
NFR-2	PR-AUC 0.80 or more and recall 0.90 or more on test; Silhouette 0.30 or more.
NFR-3	No leakage: features use past data only, and the test split is used once per model family.
NFR-4	Reproducible within 0.01 with seed 42 and the pinned library versions.
NFR-5	Anonymised data only; uploads are not saved; no keys in the repository.
NFR-6	A new user finds a high-risk customer in 3 minutes or less.
NFR-7	Reasons are written in plain words, not only as feature names.
NFR-8	Runs on Windows, macOS and Linux with relative paths.

4. Software Architecture

The system has two halves. The offline half cleans the data, builds the features, trains the models and saves them. The online half is the dashboard: it reads the saved files and shows results. The two halves share files only, so a demo can never change the model behind a reported number.

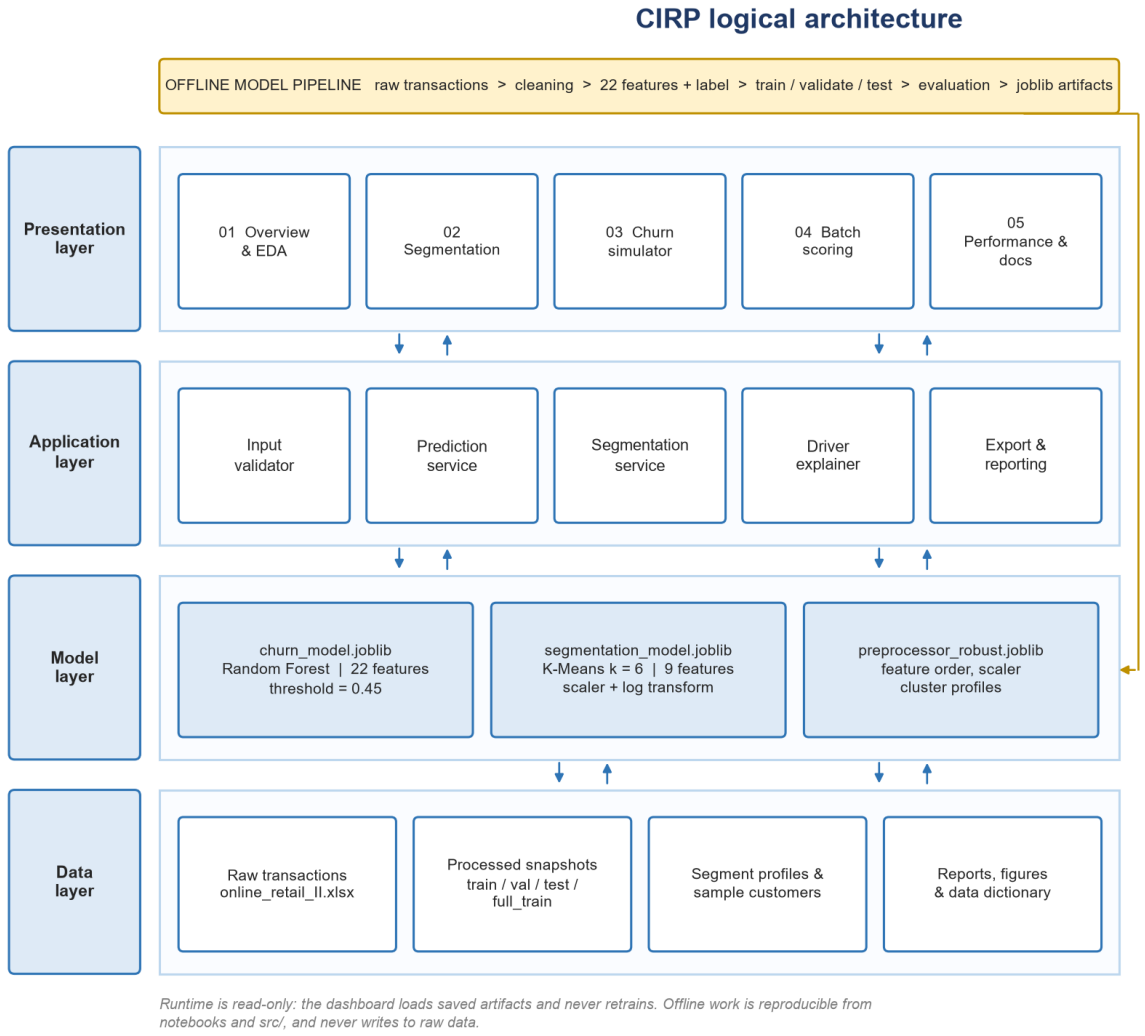


Figure 2. Architecture: offline pipeline and online dashboard

4.1 Main components

Table 5. Main components

Component	What it does
app.py	Chooses the page, keeps the session state and shows the model status.
Five page files	Draw the overview, segments, simulator, batch and performance pages.
churn_predictor.py	Loads the churn model file, checks the 22 features, returns a probability and a label.
segment_predictor.py	Loads the segmentation file, applies the transform, returns a cluster and its profile.
Model files	Two joblib files that hold the models, the threshold and the feature list.
Data files	processed CSV files for train, validation and test, plus the cluster profiles.

4.2 Design rules

- **No leakage.** A feature never looks at data after the snapshot date, and the scaler is fitted on training data only.
- **Read-only models.** The dashboard loads model files and never trains or overwrites them.
- **One feature list.** The 22 names and their order are stored with the model, not copied into the app.
- **Clear errors.** A missing value or a missing model file produces a readable message, never a made-up score.

5. Database Design

The delivered project uses CSV files and two model files, not a database server. The diagram below is the logical design. It also shows how the same design would look in PostgreSQL if the project moved to a server.

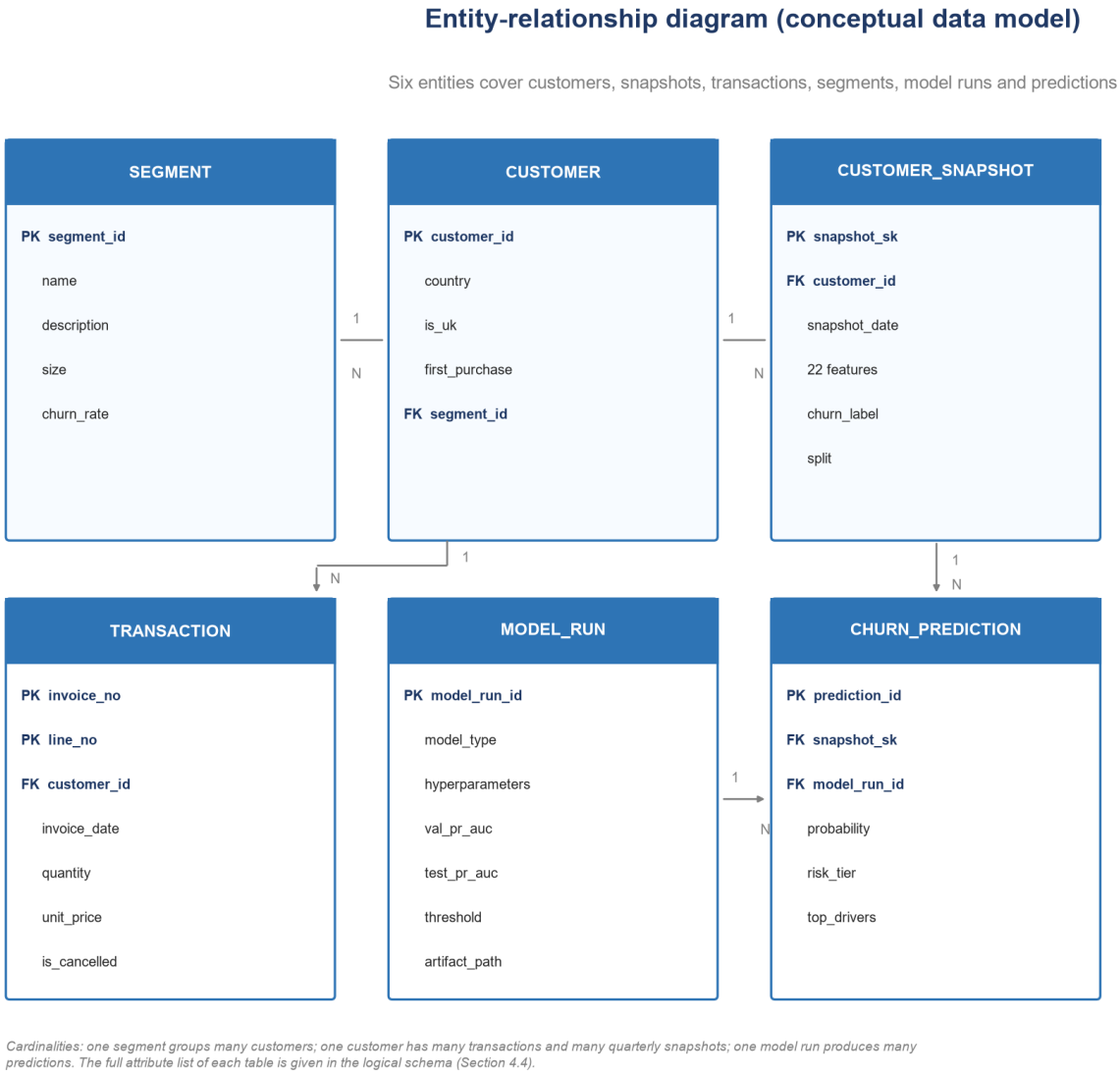


Figure 3. Entity-relationship diagram

A customer has one record per quarterly snapshot. Each record holds the 22 features and the churn label. A prediction uses one record and one model run, so it can always be repeated.

5.1 Main tables

Table 6. Main tables and where they are stored

Table	Key	Main fields	Rows	Stored as
customer	customer_id	country, is_uk, first_purchase, segment_id	5,253	Derived from the data files
customer_snapshot	snapshot_id	customer_id, snapshot_date, 22 features, churn_label, split	22,372	train.csv, val.csv, test.csv
transaction	invoice_no + line_no	customer_id, invoice_date, stock_code, quantity, unit_price	about 1.0 M	Raw workbook, read only
segment	segment_id	name, description, suggested_action, size, churn_rate	6	cluster_profiles.csv

Table	Key	Main fields	Rows	Stored as
model_run	model_run_id	model_type, hyperparameters, val_pr_auc, test_pr_auc, threshold	about 40	Experiment tables
churn_prediction	prediction_id	snapshot_id, model_run_id, probability, risk_tier, top_drivers	not stored	Computed when needed

5.2 The 22 features

Every model uses the same 22 features: recency, frequency, monetary, average order value, order value spread, tenure, gaps between orders, recency-to-gap ratio, single-order flag, number of products, basket size, unit price, return rate, orders in the last 30, 60 and 90 days, spend in the last 90 days, spend trend, frequency trend, UK flag and weekend share. The data dictionary gives the full definition and the range of each one.

6. Data Flow

6.1 Context diagram

The dataset is the only outside source of data. Users send requests and get results, and nothing else crosses the system boundary.

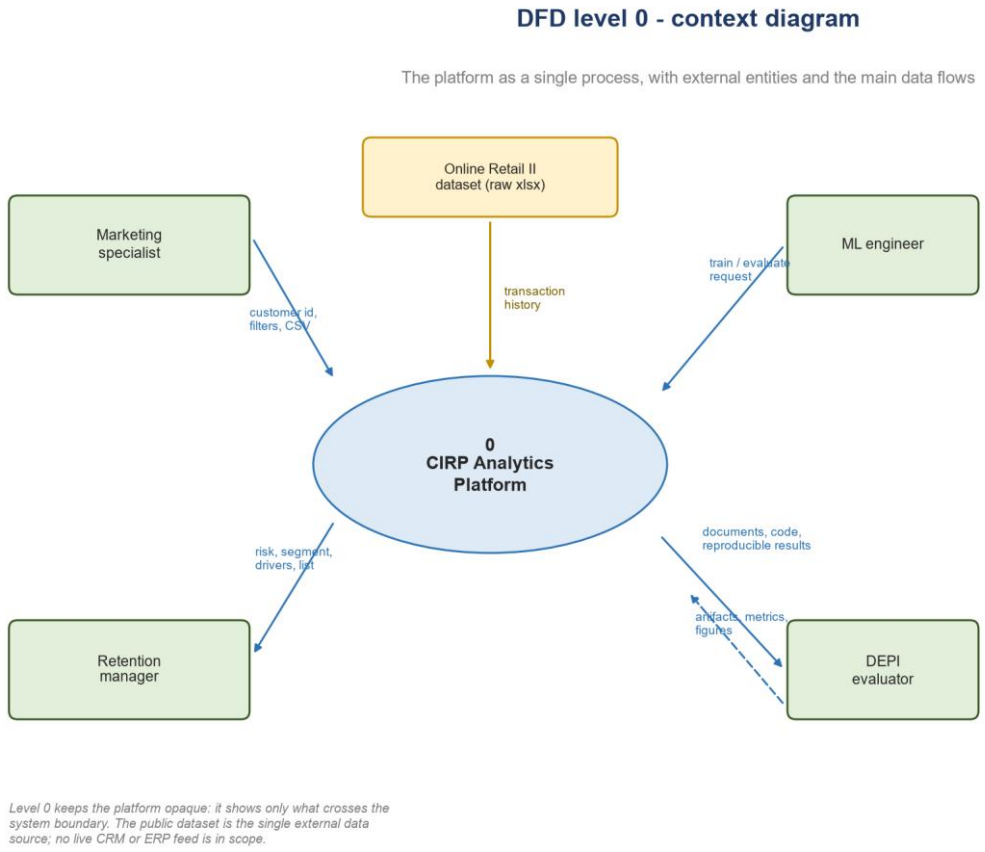


Figure 4. Context diagram (DFD level 0)

6.2 Detailed data flow

Level 1 opens the system into five processes. P1 to P3 run offline. P4 and P5 run on every request. The model files are the only link between the two sides.

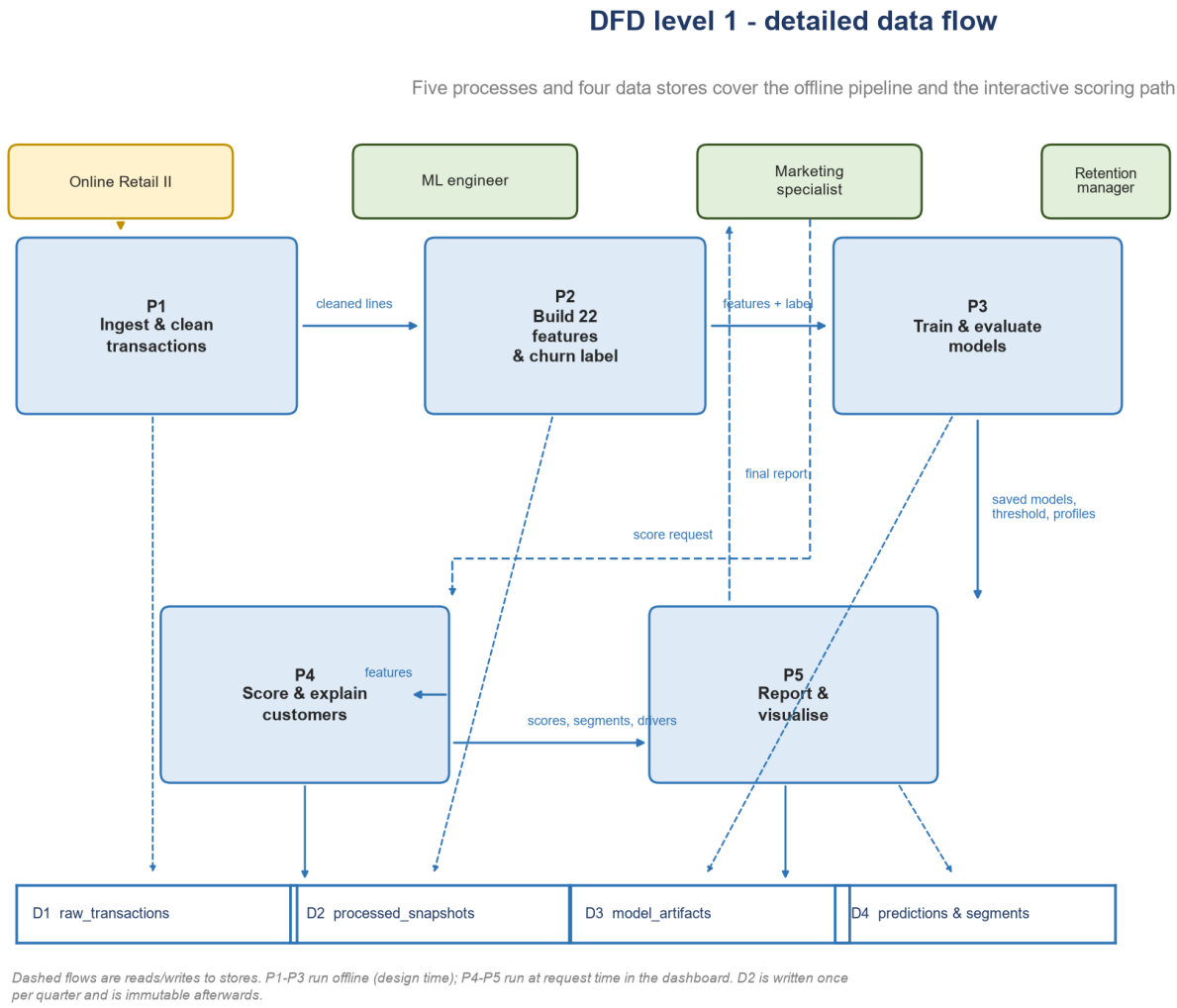


Figure 5. Detailed data flow (DFD level 1)

Table 7. Processes of the data flow

ID	Process	Input	Output
P1	Clean the transactions	Raw workbook	Clean transaction table
P2	Build features and labels	Clean transactions	Snapshot dataset
P3	Train and evaluate	Snapshot dataset	Model files and metrics
P4	Score and explain a customer	Features and model files	Probability, reasons and segment
P5	Show results and export	Scores and profiles	Dashboard pages and CSV

7. Behaviour Diagrams

7.1 Sequence diagram: score one customer

The page checks the input first, then calls the model. The feature order and the threshold come from the model file, not from the page.

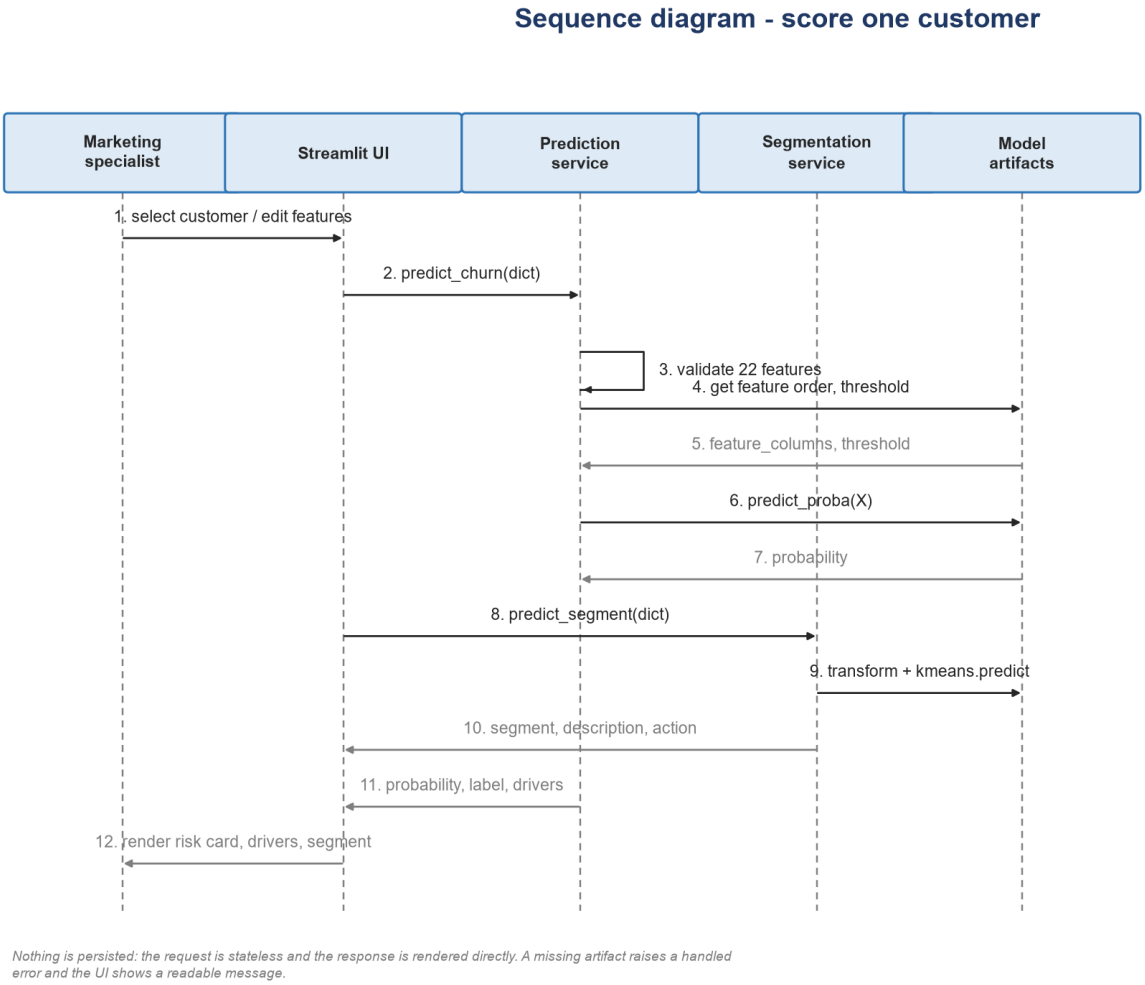


Figure 6. Sequence diagram: one customer

7.2 Training order

Training follows the same order every time:

1. Load the training split and fit the scaler on those rows only.
2. Train the three baseline models and compare them on validation.
3. Tune the Random Forest and choose the threshold on validation.
4. Refit the chosen model on train plus validation, then score test once.

7.3 Activity diagram

The single-customer path and the batch path both end in a read-only result. A wrong input returns to the form or to the upload, and no model is called.

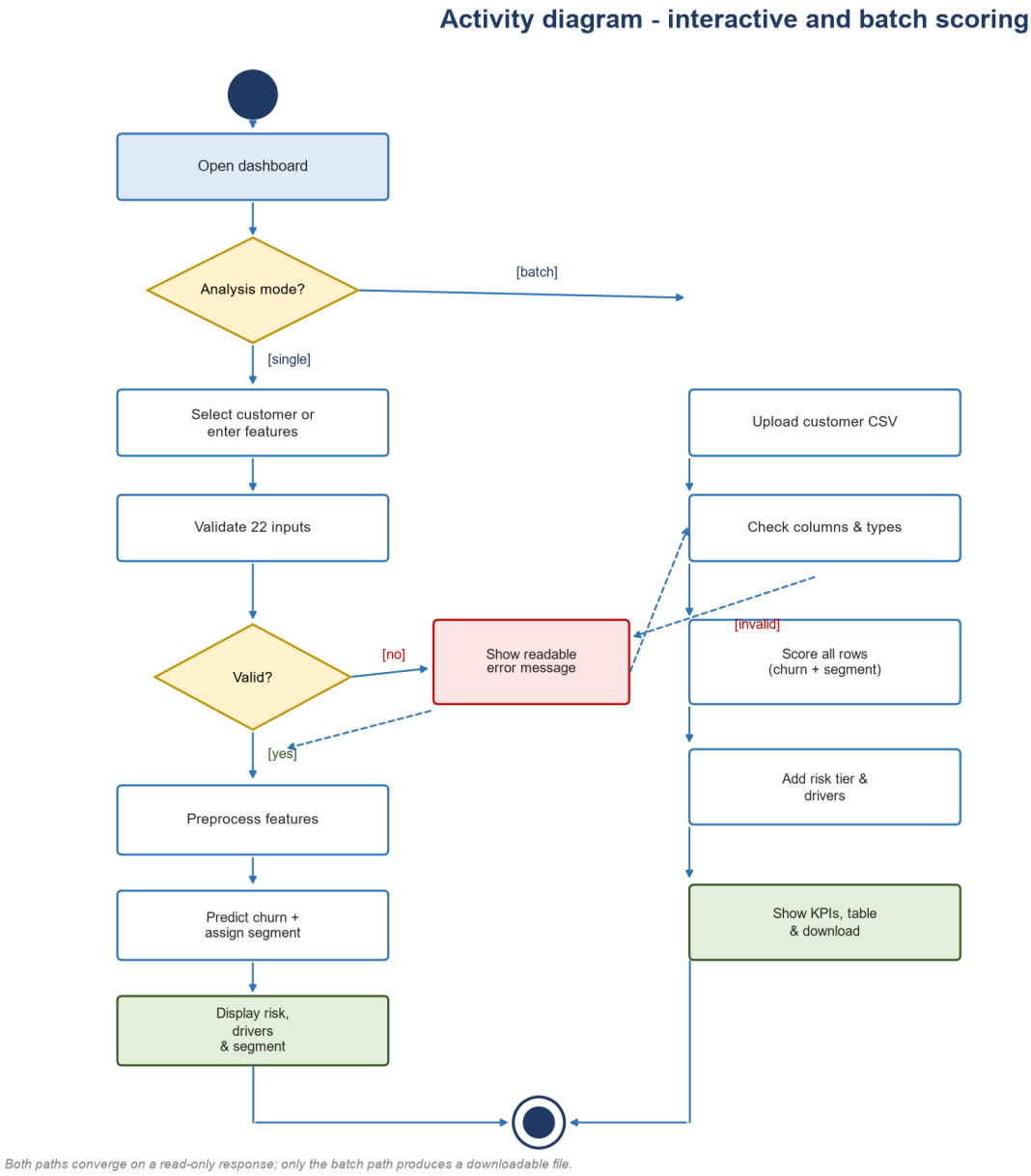


Figure 7. Activity diagram: single and batch scoring

7.4 State diagram

The states describe the customer lifecycle behind the churn label: new, active, at risk, dormant, churned and reactivated. The platform does not store these states; they explain the label to business users.

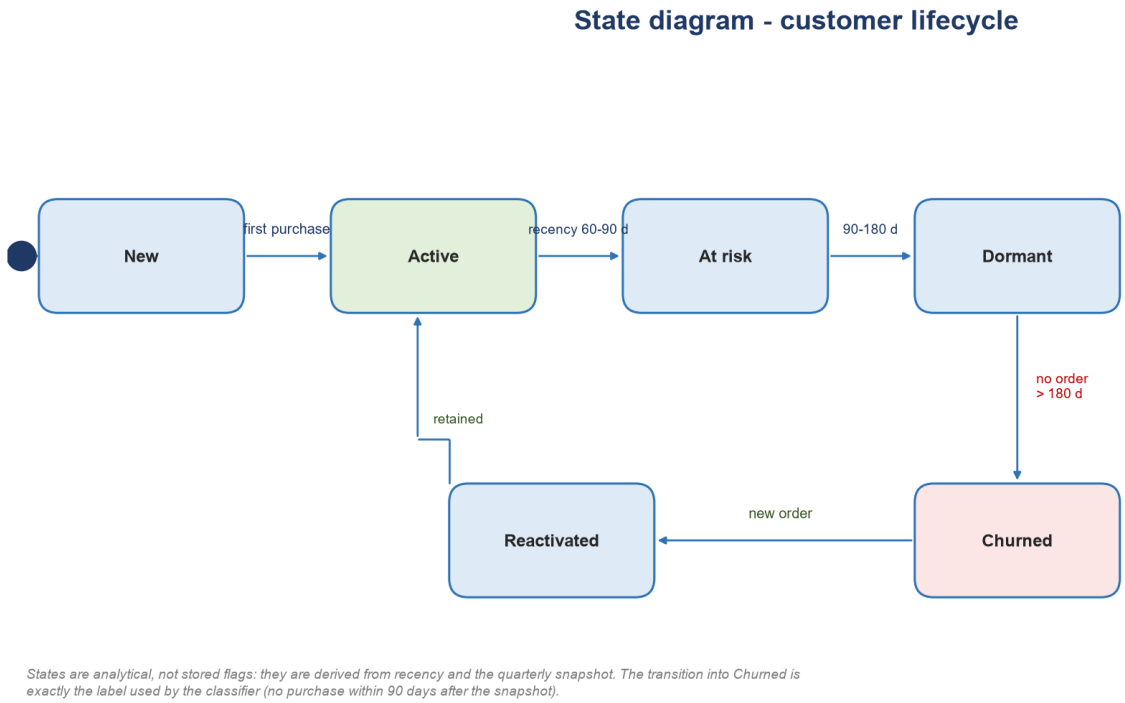


Figure 8. State diagram: customer lifecycle

7.5 Class diagram

Only the runtime is shown: the pages, one scoring service, two model wrappers and two small data classes. The offline code is made of scripts, so it is not part of this diagram.

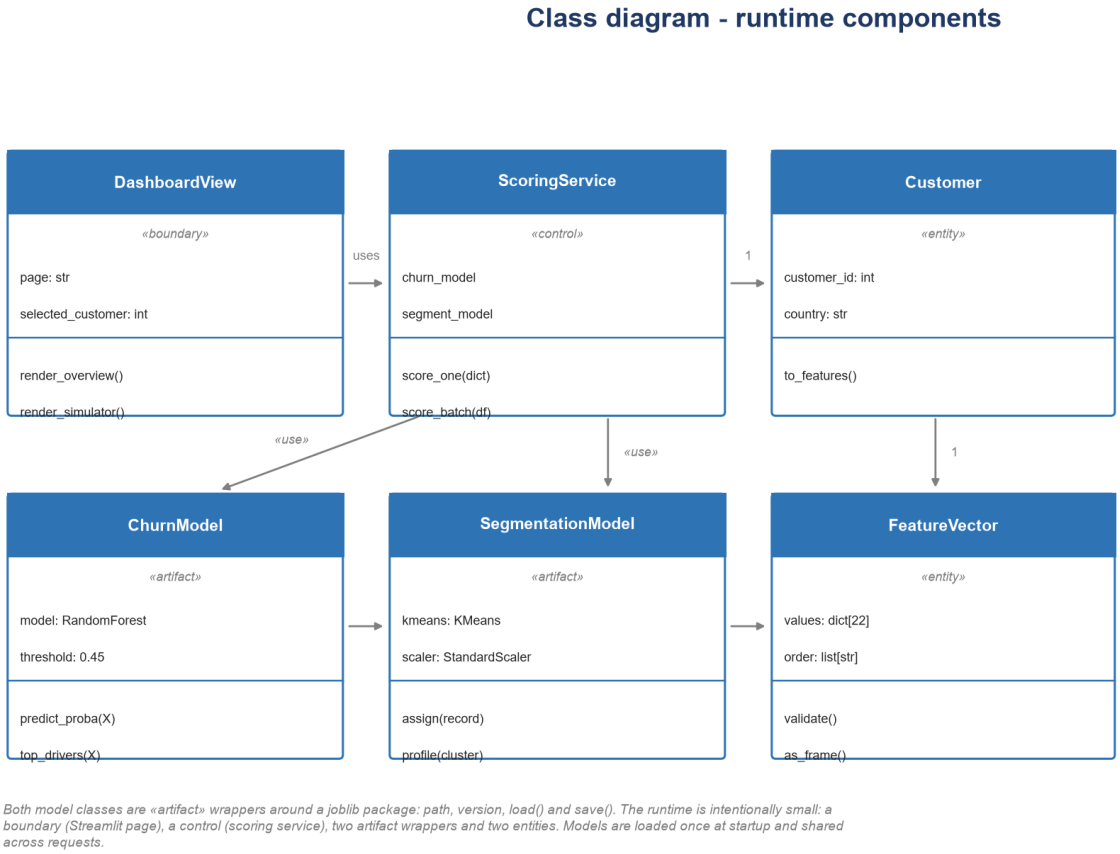


Figure 9. Class diagram: runtime components

8. Screens and UI Guidelines

The dashboard is built for a business user with three minutes and no machine-learning background. Each page answers one question and shows the answer first.

- Five flat pages in one sidebar, with no sub-menus and no login in the local version.
- Red, amber and green always mean high, medium and low risk. Blue is used for navigation.
- The biggest element on each page is the decision, not the chart behind it.
- No page can retrain, overwrite or delete a model.

8.1 Overview page

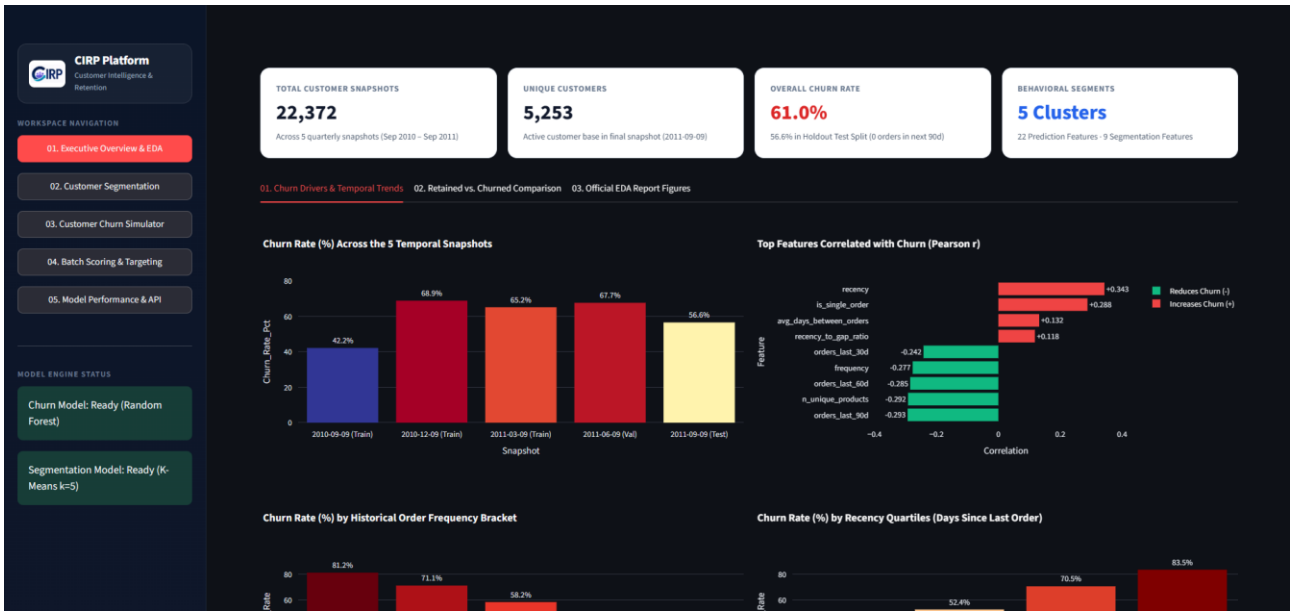


Figure 10. Wireframe: overview and EDA

Four KPI cards come first: customers, churn rate, average recency and revenue at risk. Two charts and a small segment table follow.

8.2 Simulator page

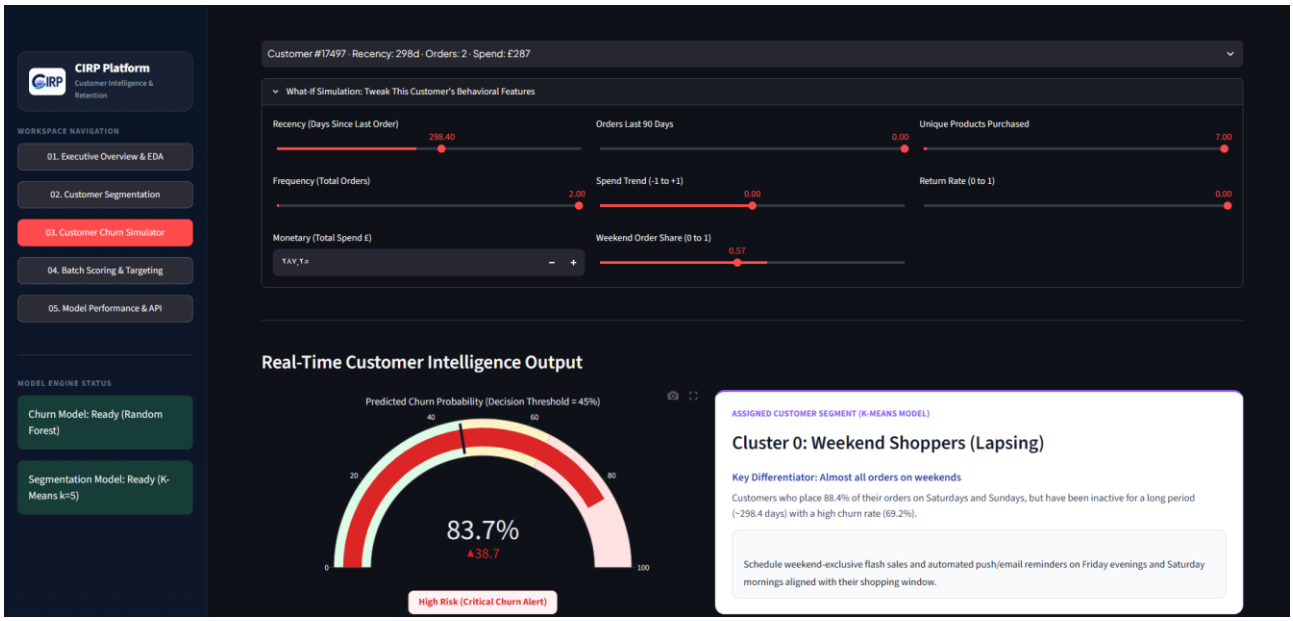


Figure 11. Wireframe: churn simulator

The left side is the input form. The right side is the answer: the probability, a risk label, the top three reasons and the segment card.

8.3 Batch page

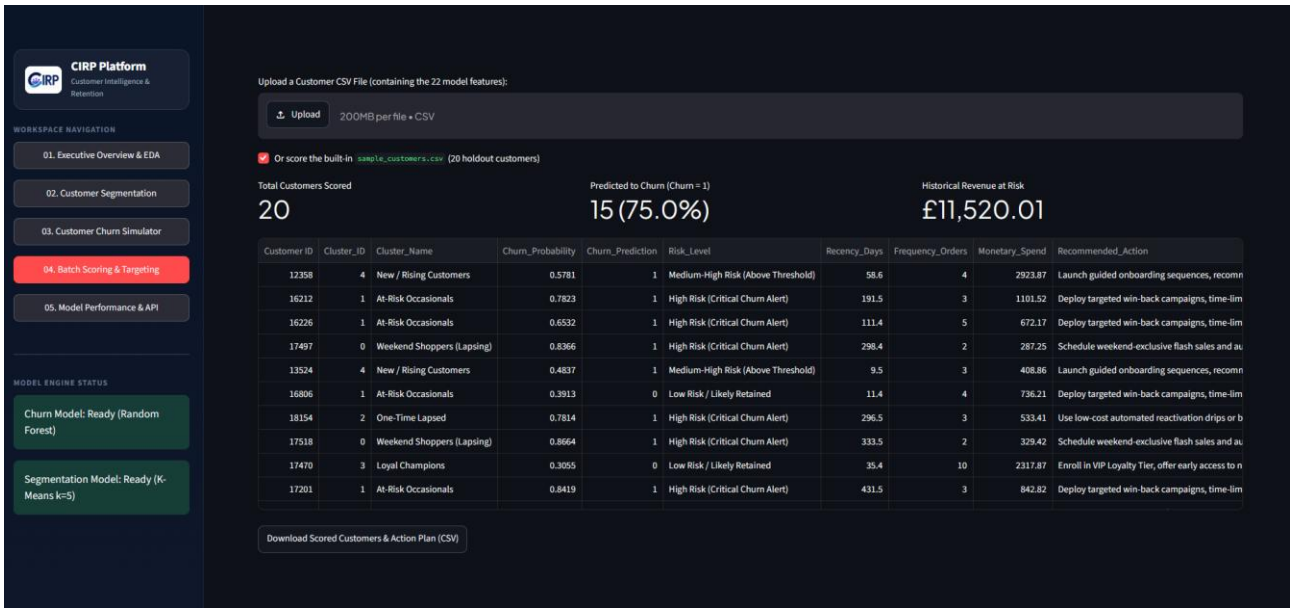


Figure 12. Wireframe: batch scoring

The user drops a CSV on the page, sees three summary numbers and a sortable table, then downloads the result. A file with wrong columns is rejected before any row is scored.

8.4 Colours and fonts

Table 8. Colour and font rules

Item	Value	Used for
Navy	#0F172A	Sidebar and main text
Blue	#2563EB	Buttons, links and the selected page
Red	#DC2626	High risk and revenue at risk
Amber	#D97706	Medium risk
Green	#16A34A	Low risk and good news
Fonts	Plus Jakarta Sans, JetBrains Mono	Headings and body; numbers and IDs

9. Technology and Deployment

9.1 Technology stack

Table 9. Technology stack

Layer	Technology	Used for
Language	Python 3.10+	All code
Data	pandas, NumPy, openpyxl	Reading, cleaning and building the snapshots
Machine learning	scikit-learn 1.9.0	Models, K-Means, scaler and metrics
Deep learning	PyTorch, pytorch-tabnet	The isolated MLP, FT-Transformer and TabNet comparison
Charts	matplotlib, seaborn	EDA, ROC and PR curves, confusion matrices
Dashboard	Streamlit	The five-page application
Charts in the app	Plotly	Interactive charts with the project colours
Saving models	joblib	The two model files
Version control	Git and GitHub	Branches, pull requests and documents
Documents	Word, Markdown, PowerPoint	The DEPI documents and the presentation

9.2 Deployment diagram

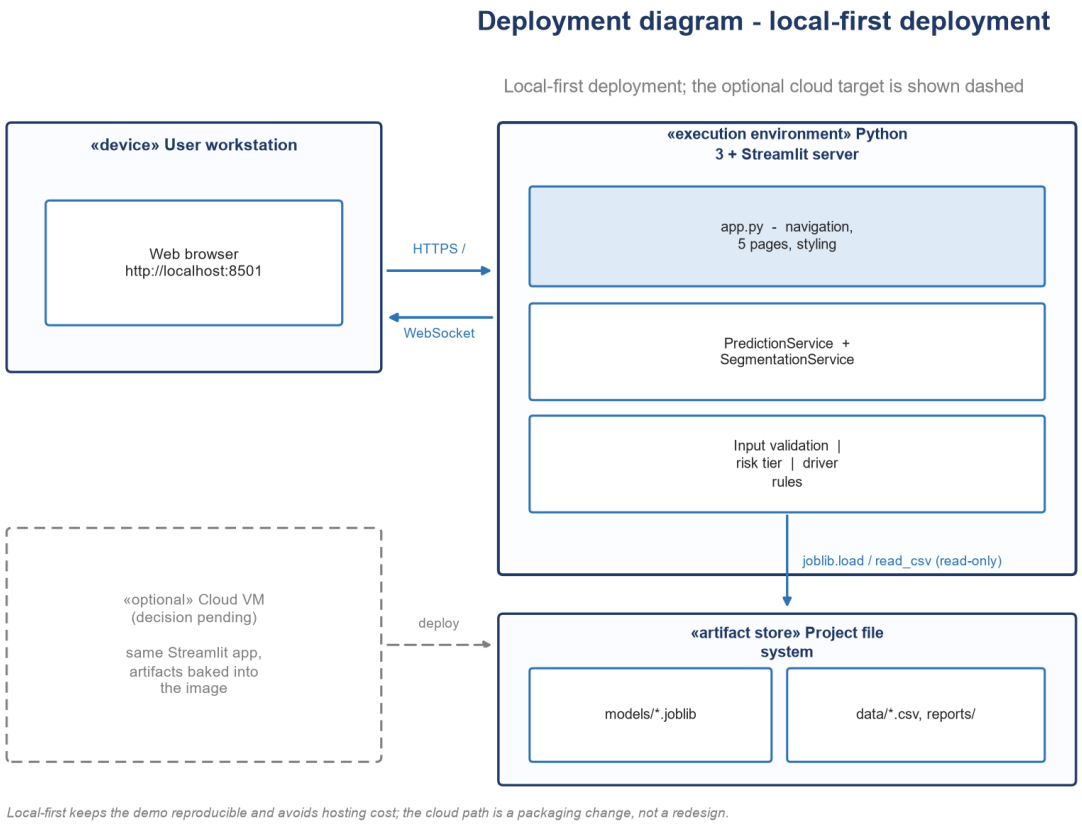


Figure 13. Deployment diagram

The default setup is one laptop and one Python process. The browser talks to the Streamlit server, and the server reads the local model and data files. A cloud machine is optional and runs the same files.

9.3 Component diagram

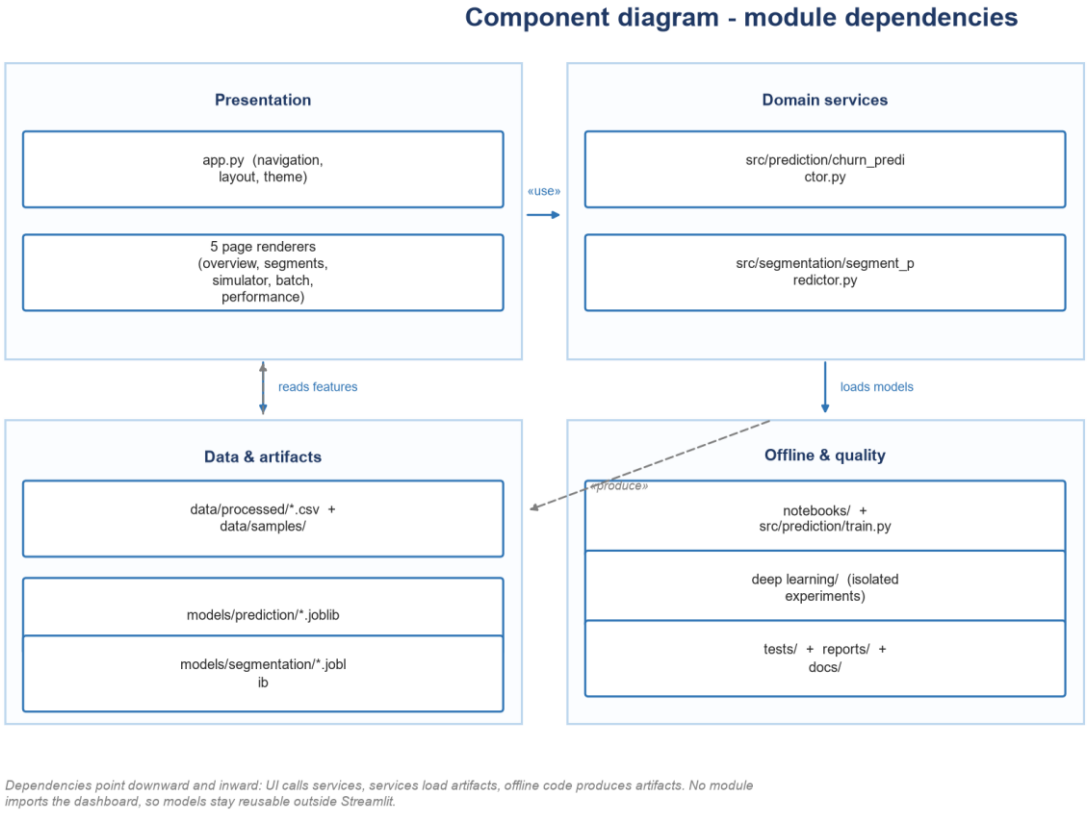


Figure 14. Component diagram

Arrows point from the pages to the services and from the services to the saved files. Model code never imports the dashboard, so it can also be used from a notebook or a test script.

9.4 How to run

1. Install the packages from `requirements.txt` in a Python environment.
2. Put the two model files in `models/` and the processed CSV files in `data/processed/`.
3. Run `streamlit run app/app.py` from the project folder.
4. Open `http://localhost:8501` and check that the sidebar shows both models as ready.

10. Testing and Delivery

10.1 Test levels

Table 10. Test levels

Level	What we test	How
Unit	Feature arithmetic, threshold, risk level, input checks	Small tests on fake data
Integration	Model loading, feature order, same score after reload	Two smoke scripts
Model validation	Split dates, leakage checks, metrics, threshold	Evaluation script on validation, one test run
User acceptance	The five pages and the CSV export	Ten scripted scenarios with a non-team user

10.2 Model results

Table 11. Results of the production model and the best deep-learning model

Model	Split	PR-AUC	ROC-AUC	Recall	Precision	F1	Accuracy
Random Forest	Validation	0.8911	0.8217	-	-	-	78.81%

Model	Split	PR-AUC	ROC-AUC	Recall	Precision	F1	Accuracy
Random Forest	Test	0.8224	0.8010	95.87%	66.20%	78.32%	69.94%
FT-Transformer	Validation	0.8940	0.8241	-	-	-	78.61%
FT-Transformer	Test	0.8082	-	93.14%	68.49%	-	71.84%

The deep-learning model is slightly better on validation but worse on test, and it misses more churners. Because the project cares most about finding churners, the Random Forest stays in production. The full comparison is in the deep-learning report.

10.3 User acceptance scenarios

Table 12. User acceptance tests

ID	Scenario	Expected result
UAT-01	Open the app with both model files present.	Both models show as ready and the overview loads.
UAT-02	Score a known customer.	The same probability as the saved test row.
UAT-03	Enter an invalid value.	A clear message names the field and no score appears.
UAT-04	Upload a valid CSV.	Every row is scored and the download works.
UAT-05	Upload a CSV with missing columns.	The missing columns are listed and nothing is scored.

10.4 Delivery

- The release is the tagged repository plus the frozen data and model files.
- A release needs zero critical defects and all acceptance tests passing.
- Rollback means restoring the previous model file and its metrics file.
- A future version can add an API, a PostgreSQL database and a login, without changing the models.